

 REAL-WORLD PROBLEM SCENARIO

Location: Jaipur-Ajmer Expressway (Coordinates: `26.9124, 75.7873`)

Incident: High-speed highway collision with two trapped victims suffering traumatic head and chest injuries.

Clinical Requirement: Polytrauma requiring **Advanced Life Support (ALS)** with mechanical ventilation, rapid road transit, and admission to an Apex Level-1 Trauma Emergency Department.

1. Why Traditional 911 / 108 EMS Systems Fail This Scenario

Legacy emergency dispatch systems rely on naive Euclidean (straight-line) distance and assign whatever vehicle is closest, creating severe systemic breakdowns:

FAILURE DIMENSION	TRADITIONAL EMS BEHAVIOR	H8 CAPABILITY-AWARE SOLUTION
1. Vehicle Capability Match	FATAL ERROR Dispatches the closest unit, which is a Basic Life Support (<code>BLS</code>) van without ventilator or doctor.	SOLVED <code>DispatchScorer</code> checks <code>requiresAls=true</code> . BLS units are heavily penalized; ALS unit <code>AMB-01</code> ranks #1.
2. City Coverage Vacuum	VACUUM CREATED Pulls the last remaining ambulance from South Jaipur, leaving 60,000 citizens with zero emergency coverage.	SOLVED <code>redeployment-service</code> runs MEXCLP algorithms and proactively orders idle unit <code>AMB-02</code> to reposition.
3. Receiving ED Selection	ED OVERLOAD Transports to the nearest clinic, which is on diversion and has 0 trauma ICU beds. 45-minute delay ensues.	SOLVED <code>DestinationRanker</code> evaluates live bed capacity, diversion flags, and specialist capabilities to route directly to SMS Apex Hospital.
4. Concurrency & Liability	RACE CONDITIONS Two call-takers double-book the same ambulance. Paper logs lack legal audit trail.	SOLVED Zero-race conditional SQL updates (<code>'status = AVAILABLE'</code>) and SHA-256 Merkle hash chain auditability.

2. System Architecture Diagram

The system is built on a clean multi-module architecture: a pure Java 21 algorithmic engine (`common`) surrounded by event-driven Spring Boot 3 microservices, Kafka, Redis, and PostGIS:



3. Visual Mindmap of the Web Application

The Dispatcher Command tactical dashboard maps user actions directly to algorithmic calculations and atomic database events:

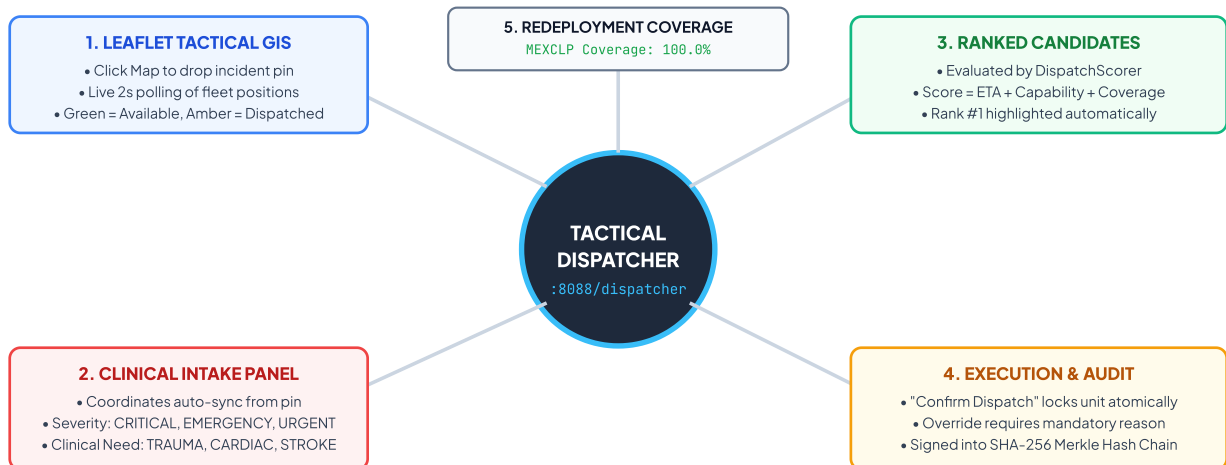


FIGURE 2: MINDMAP OF DISPATCHER DASHBOARD MODULES & ALGORITHMIC TRIGGERS

4. Step-by-Step Incident Lifecycle Workflow

1 Emergency Call Intake

Call received at central intake. Caller phone number is hashed with a secret salt (`CallerHashUtil.java`). Raw citizen PII is never stored (Rule #6). Incident location is saved as PostGIS Point in schema `incident` .

2 Multi-Factor Candidate Scoring

`DispatchScorer` evaluates all available ambulances. Computes turn-by-turn road travel time via GraphHopper, capability penalty for non-ALS units, and freshness penalty for stale GPS pings (>60s). Ranks best units in real time.

3 Atomic Zero-Race Reservation

Dispatcher hits "**Confirm Dispatch**". Database executes conditional SQL: `UPDATE ambulance_unit SET status = 'DISPATCHED' WHERE id = :id AND status = 'AVAILABLE'` . Eliminates double-booking bugs. Decision hashed to Merkle chain.

4 Proactive City Protection

The moment unit departs, `redeployment-service` runs the MEXCLP coverage algorithm. If a suburban zone drops below safety thresholds, it triggers a repositioning order for an idle unit to preserve citywide safety.

5 Live Device GPS Telemetry

Crew taps "  **Transmit My Device Live GPS** " on mobile phone (`/crew/index.html`). High-accuracy coordinates stream every 2s. Dispatcher map updates the ambulance marker sliding along roads toward patient.

6 Hospital ED Matching & Handover

`DestinationRanker` scores all receiving hospitals by road transit time, free trauma beds, and diversion states. Top hospital is pre-alerted via SSE. Paramedics complete digital handover, resetting unit status to `AVAILABLE` .

Verification Status: Microservices active on ports 8080–8087. Docker infrastructure (PostGIS 5432, Kafka 9092, Redis 6379, Keycloak 8180) healthy. Web portals live on port 8088.